Founding Reliability Engineer: 1-Week Live Assessment

Nov 1, 2026 · @Vaibhav Goyal

Overview

Over 7 days, you'll take a fragile platform that serves one client and make it ready to run 8 clients safely, using your own cloud account. Then you'll walk us through it and handle a live incident with us.

Day	What happens
Day 0 ([start date])	30-minute kickoff call. We go through this brief and answer your questions.
Days 1–6	You build. An optional 20-minute check-in is available on Day 3.
Day 7 ([deadline date, time and time zone])	You submit your work (see Submission).
Days 8–10	2-hour live session: walkthrough, live incident drill, and a talk about your 90-day plan.

Time. We expect about 20–25 hours of work spread across the week. The assessment is built so that nobody finishes everything. What you choose to do first, and how you explain what you left out, is part of what we assess.

The role. This is a founding reliability engineer role. You'll own the platform's uptime, security and operations, automate how we bring on clients, and build the reliability function as we grow. The tasks below are close to the work you'd do in your first months.

Fairness. Every candidate gets the same brief, the same resources, the same answers to questions and the same live scenarios. If you need an accommodation, such as more time or a different format for the live session, contact [accommodations contact].

The scenario

You've just joined Ledgerline, a fictional B2B SaaS company, as its founding reliability engineer. Sales has signed 7 new clients who go live over the next 6 months, and the platform as it stands can't take them.

What exists today

- One API service (the reference app in Resources) and one Postgres database, both on a single virtual machine.
- Deploys happen by SSH-ing into the machine and running `docker compose up`.
- Secrets live in a `.env` file on the machine and in the code's defaults.
- Every client is identified by an `X-Tenant-Id` header. Nothing checks that the caller is allowed to use it.
- There is no monitoring, no alerting, no tested backup and no runbook. The founders find out about outages from the client.
- One client, Acme Retail, is live on this setup.

The clients coming on board

Client	Go-live	Tier	What they require
Acme Retail	Live now	Shared	Must be moved without data loss and with under 5 minutes of downtime
Brightpath Learning	Month 1	Shared	Standard onboarding
Cobalt Logistics	Month 2	Shared	Heavy reporting use: runs large reports during business hours
Delta Finance	Month 2	Dedicated	Regulated. Its own database, encryption at rest with its own key, audit log of every admin action, answers to its security questionnaire before signing
Evergreen Health	Month 3	Dedicated	Sensitive data. Backups restorable within 1 hour, losing no more than 15 minutes of data
Fjord Media	Month 4	Shared	Traffic spikes 10x during campaigns
Granite Insurance	Month 5	Dedicated	Data must stay in a different region from the other clients
Harbor Foods	Month 6	Shared	Standard onboarding

Constraints

- Ledgerline has no other reliability or platform engineer. Whatever you build, a future hire must be able to run from your documentation alone.
- The company is small. Cost matters, and you'll be asked to justify what the setup costs per client.

- Adding a client should take hours, not weeks, and should not need you to be online.

The tasks

There are 6 tasks and a set of stretch goals. They aren't in order of importance, and you won't finish all of them. Choose where to go deep, and tell us why in your README.

Task 1: Build the platform as code

1. Provision everything from an empty cloud account with infrastructure as code (Terraform, OpenTofu, Pulumi, CDK or similar). One documented command should bring up the whole platform.
2. Run two environments, staging and production, from the same code.
3. Make onboarding a client one repeatable step, for example `make onboard TENANT=brightpath TIER=shared`. It should support both tiers:
   - **Shared:** the client's data lives in a pooled database, kept apart from other clients.
   - **Dedicated:** the client gets its own database, and its own encryption key.
4. Show it working with at least 3 tenants: Acme Retail and Brightpath (shared), and Delta Finance (dedicated).
5. Move Acme Retail's existing data onto the new platform. Describe how you'd do it in production with under 5 minutes of downtime, and show it on the seed data.

Task 2: Secure it and keep clients apart

1. The reference app has several security and reliability flaws, some deliberate. Find as many as you can, fix them, and list each one with its fix in `SECURITY.md`.
2. Replace the trust-the-header approach with real authentication, so a caller can only reach its own tenant's data.
3. Write an automated test that proves tenant A can't read or change tenant B's data, and run it in your pipeline.
4. Keep secrets in a secrets manager, never in code, images or the repo. Give every service and person the least access it needs.
5. Encrypt data at rest and in transit. Record every admin action in an audit log that the admin can't edit.
6. Answer Delta Finance's security questionnaire (in Resources) as you would send it to the client. Answer honestly: "not yet, planned for month 2" is a fine answer.

Task 3: Make it observable

1. Collect metrics, logs and traces, all tagged by tenant.

2. Build one dashboard that shows platform health and can be filtered to one tenant.
3. Define service level objectives (SLOs) for availability and latency, and explain why you picked those numbers.
4. Set up at least 3 alerts tied to those SLOs, sent somewhere you can show us (email, Slack, a pager tool or similar). Avoid alerts that would fire every night.
5. Write a one-page error budget policy: what happens to releases and priorities when an SLO's error budget runs out.

Task 4: Ship changes safely

1. Build a pipeline that runs tests, a dependency and container scan, and the tenant isolation test on every change.
2. Deploy without downtime, and roll back with one step.
3. Make database schema changes safe to run while the app is serving traffic.
4. Roll a release out to one tenant first, then to the rest.

Task 5: Make it hold up under pressure

1. **Backups:** take automated backups and prove a restore. Restore Evergreen Health's database into a fresh instance, time it, and record how much data you'd lose (target: restored within 1 hour, under 15 minutes of data lost).
2. **Load:** run the provided load test against production. Target: 200 requests per second for 10 minutes across 3 tenants, with 95% of `/records` requests answered in under 300 ms.
3. **Noisy neighbour:** while the load test runs, have Cobalt Logistics hammer `/reports/heavy`. Other tenants' latency must stay within target. Show the before and after.
4. Write up what broke first under load, and what you changed.

Task 6: Run it and lead it

1. Write 3 runbooks a new engineer could follow at 2am: onboarding a client, restoring a database, and responding to an incident.
2. Write a blameless post-incident review of the worst problem you hit during the week.
3. Estimate the monthly cost for one shared client and one dedicated client, and show what drives each number.
4. Write a 90-day plan as Ledgerline's founding reliability engineer, at most 2 pages. Cover what you'd do first, what you'd put off, the risks you see, and who you'd hire first and when.

Stretch goals (only after the core is solid)

- Deploy Granite Insurance into a second region, using the same code.
- Attribute cloud cost to each tenant through tags or labels, and show it.
- Enforce security rules as code (policy as code) in the pipeline.
- Rotate a secret automatically with no downtime.
- Handle Fjord Media's 10x spike with autoscaling, and show it in a load test.

Resources

Everything you need is below: the reference app, a seed script, a load test and Delta Finance's security questionnaire. Copy the files into a new repository to start.

Reference app

This is Ledgerline's API as it runs today. It works, and it has flaws. You may rewrite it in another language or framework, as long as the endpoints behave the same way for clients.

`main.py`

```
import os
import random
import time

import psycopg
from fastapi import FastAPI, Header
from psycopg.types.json import Jsonb

DATABASE_URL = os.getenv(
    "DATABASE_URL", "postgresql://admin:admin123@db:5432/ledgerline"
)
ADMIN_TOKEN = os.getenv("ADMIN_TOKEN", "letmein")

app = FastAPI()


def get_conn():
    return psycopg.connect(DATABASE_URL, autocommit=True)


@app.on_event("startup")
def create_schema():
    with get_conn() as conn:
        conn.execute(
            """
```

```
            CREATE TABLE IF NOT EXISTS records (
                id SERIAL PRIMARY KEY,
                tenant_id TEXT NOT NULL,
                payload JSONB NOT NULL,
                created_at TIMESTAMPTZ NOT NULL DEFAULT now()
            )
            """
        )


@app.get("/health")
def health():
    return {"status": "ok"}


@app.post("/records")
def create_record(payload: dict, x_tenant_id: str = Header()):
    with get_conn() as conn:
        row = conn.execute(
            "INSERT INTO records (tenant_id, payload) VALUES (%s, %s)
RETURNING id",
            (x_tenant_id, Jsonb(payload)),
        ).fetchone()
    return {"id": row[0]}


@app.get("/records")
def list_records(x_tenant_id: str = Header(), search: str = ""):
    query = (
        f"SELECT id, payload, created_at FROM records "
        f"WHERE tenant_id = '{x_tenant_id}' AND payload::text ILIKE '%
{search}%' "
        f"ORDER BY id DESC"
    )
    with get_conn() as conn:
        rows = conn.execute(query).fetchall()
    return [{"id": r[0], "payload": r[1], "created_at": r[2]} for r in rows]


@app.get("/records/{record_id}")
def get_record(record_id: int):
    with get_conn() as conn:
        row = conn.execute(
            "SELECT id, tenant_id, payload FROM records WHERE id = %s",
(record_id,)
        ).fetchone()
```

```
    return {"id": row[0], "tenant_id": row[1], "payload": row[2]}


@app.get("/reports/heavy")
def heavy_report(x_tenant_id: str = Header()):
    time.sleep(random.uniform(0.5, 3.0))
    with get_conn() as conn:
        count = conn.execute(
            "SELECT count(*) FROM records r1 JOIN records r2 "
            "ON r1.tenant_id = r2.tenant_id WHERE r1.tenant_id = %s",
            (x_tenant_id,),
        ).fetchone()[0]
    return {"tenant": x_tenant_id, "pairs": count}


@app.delete("/admin/tenants/{tenant_id}")
def delete_tenant(tenant_id: str, token: str = ""):
    if token != ADMIN_TOKEN:
        return {"error": "unauthorized"}
    with get_conn() as conn:
        conn.execute("DELETE FROM records WHERE tenant_id = %s",
(tenant_id,))
    print(f"deleted tenant {tenant_id} using token {token}")
    return {"deleted": tenant_id}
```

`requirements.txt`

```
fastapi==0.115.0
uvicorn[standard]==0.30.6
psycopg[binary]==3.2.3
```

`Dockerfile`

```
FROM python:3.12
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD uvicorn main:app --host 0.0.0.0 --port 8000
```

`docker-compose.yml`

```
services:
  api:
    build: .
    ports:
      - "8000:8000"
```

```
    environment:
      DATABASE_URL: postgresql://admin:admin123@db:5432/ledgerline
    depends_on:
      - db
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: admin123
      POSTGRES_DB: ledgerline
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
volumes:
  pgdata:
```

Seed script

Run `docker compose up` once so the app creates its table, then load Acme Retail's existing data with `python seed.py acme 20000`. Use `python seed.py cobalt 5000` to give Cobalt Logistics enough data for heavy reports.

`seed.py`

```
import os
import random
import sys

import psycopg
from psycopg.types.json import Jsonb

DATABASE_URL = os.getenv(
    "DATABASE_URL", "postgresql://admin:admin123@localhost:5432/ledgerline"
)
tenant, count = sys.argv[1], int(sys.argv[2])

rows = [
    (
        tenant,
        Jsonb(
            {
                "order": i,
                "amount": round(random.uniform(5, 500), 2),
                "sku": f"SKU-{random.randint(1, 999):03d}",
            }
```

```
        ),
    )
    for i in range(count)
]

with psycopg.connect(DATABASE_URL) as conn:
    with conn.cursor() as cur:
        cur.executemany(
            "INSERT INTO records (tenant_id, payload) VALUES (%s, %s)", rows
        )
    conn.commit()
print(f"seeded {count} records for {tenant}")
```

Load test

The test uses k6. Change the headers to match the authentication you build in Task 2. Run it with:

`k6 run -e BASE_URL=https://your-endpoint -e TENANTS=acme:KEY,brightpath:KEY,delta:KEY load.js`

Add `-e NOISY=1 -e NOISY_KEY=KEY` to have Cobalt Logistics run heavy reports alongside the main traffic (Task 5).

`load.js`

```
import http from "k6/http";
import { check } from "k6";

const BASE_URL = __ENV.BASE_URL;
const TENANTS = __ENV.TENANTS.split(",").map((t) => {
  const [id, key] = t.split(":");
  return { id, key };
});

const scenarios = {
  steady: {
    executor: "constant-arrival-rate",
    rate: Number(__ENV.RATE || 200),
    timeUnit: "1s",
    duration: "10m",
    preAllocatedVUs: 100,
    maxVUs: 400,
    exec: "steady",
  },
};
```

```
if (__ENV.NOISY === "1") {
  scenarios.noisy = {
    executor: "constant-vus",
    vus: 20,
    duration: "10m",
    exec: "noisy",
  };
}

export const options = {
  scenarios,
  thresholds: {
    "http_req_duration{endpoint:records}": ["p(95)<300"],
    "http_req_failed{endpoint:records}": ["rate<0.01"],
  },
};

function headersFor(id, key) {
  return {
    "Content-Type": "application/json",
    "X-Tenant-Id": id,
    Authorization: `Bearer ${key}`,
  };
}

export function steady() {
  const t = TENANTS[Math.floor(Math.random() * TENANTS.length)];
  const params = { headers: headersFor(t.id, t.key), tags: { endpoint:
"records" } };
  if (Math.random() < 0.3) {
    const body = JSON.stringify({ amount: Math.random() * 500, sku: "SKU-
LOAD" });
    const res = http.post(`${BASE_URL}/records`, body, params);
    check(res, { created: (r) => r.status === 200 || r.status === 201 });
  } else {
    const res = http.get(`${BASE_URL}/records?search=SKU-1`, params);
    check(res, { listed: (r) => r.status === 200 });
  }
}

export function noisy() {
  const params = {
    headers: headersFor("cobalt", __ENV.NOISY_KEY),
```

```
    tags: { endpoint: "heavy" },
  };
  http.get(`${BASE_URL}/reports/heavy`, params);
}
```

Delta Finance security questionnaire

Answer these in `docs/security-questionnaire.md`, written for Delta Finance's security team.

1. How is our data kept separate from other clients' data?
2. Is our data encrypted at rest and in transit? Who manages the keys, and can we have our own key?
3. Who at Ledgerline can reach our production data? How is that access granted, reviewed and removed?
4. Is multi-factor authentication required for all staff access to production?
5. How are secrets and credentials stored and rotated?
6. What do you log, how long do you keep it, and can we get an audit trail of admin actions on our account?
7. How often do you back up, how long do you keep backups, and what restore times have you tested?
8. How do you find and fix vulnerabilities in code, dependencies and infrastructure? How fast are critical issues fixed?
9. What is your incident response process, and how soon would you tell us about a security incident affecting our data?
10. In which regions is our data stored and processed? Does any third party process it?
11. How is our data deleted when the contract ends, and how do you confirm it?
12. Do you hold any security certifications, such as SOC 2 or ISO 27001, or plan to?

Tools you may use

- Any cloud provider: AWS, Google Cloud, Azure or another.
- Any infrastructure-as-code, CI/CD, container and observability tools, managed or open source.
- Managed services are welcome where they save time. Explain the trade-off in your README.

Rules

Your cloud account. Use your own account with any provider. Set a billing alert before you start. Keep total spend under [spend cap], and we'll reimburse up to [reimbursement amount] against your bill. Free tiers and small instances are fine: we're judging design and operation, not scale of spend.

Data. Use only the seed data and test data you generate. Never put real personal or customer data into your environment.

Keep it running. Leave your environment up until your live session, so we can see it working. Tear it down within 48 hours after the session, and send us a screenshot showing it's gone.

Work on your own. Don't get help from other people. Public documentation, forums and open-source code are fine; credit anything substantial you reuse.

AI tools. You may use AI coding assistants. Note in your README what you used them for. In the live session you'll need to explain and change any part of your work yourself.

Questions. Email [questions contact]. We'll reply within [response time] and share every answer with all candidates, without saying who asked. Making a reasonable assumption and writing it down is also fine.

Check-in. An optional 20-minute call on Day 3 is offered to every candidate. Use it to ask questions or test an idea. It doesn't count toward your evaluation.

Access. Never send us your cloud credentials. Share endpoints and test tenant keys only through [secure sharing method].

Submission

Submit by [deadline date, time and time zone] by giving [reviewer accounts] read access to a private Git repository. Commits after the deadline won't be reviewed, but your environment should stay running for the live session.

Repository layout

Use this layout, so every reviewer finds the same things in the same place. Add folders as you need.

```
/
├── README.md                   start here
├── SECURITY.md                 flaws found and how you fixed them
├── app/                        the API, rewritten or fixed
├── infra/                      infrastructure as code
├── .ci/ or .github/workflows   the pipeline
├── observability/              dashboards, alert rules, SLO definitions
├── loadtest/                   load test, results, before and after
└── docs/
    ├── runbooks/               onboarding, restore, incident response
    ├── decisions/              one short note per major decision
    ├── security-questionnaire.md
    ├── post-incident-review.md
    ├── cost-model.md
    └── 90-day-plan.md
```

Checklist

- [ ] `README.md` with an architecture diagram, how to deploy from an empty account, and how to onboard a tenant
- [ ] The main decisions and trade-offs, and what you'd do differently with more time
- [ ] What you didn't finish and why, in priority order
- [ ] A rough log of where your hours went
- [ ] What you used AI tools for
- [ ] Load test results, before and after your changes
- [ ] Backup restore evidence: time taken and data lost
- [ ] A screenshot of your actual cloud spend for the week
- [ ] Endpoints and test tenant keys, sent through [secure sharing method], not committed to the repo

Live session (2 hours, Days 8–10)

Part	Time	What happens
Walkthrough	30 min	You show us your platform running and walk through your main decisions.
Live incident drill	45 min	We give you a production incident on your own environment. You investigate and fix it while sharing your screen and talking us through it.

Part	Time	What happens
Deep dive	25 min	We pick parts of your code and documentation and ask you to explain or change them.
90-day plan	20 min	We discuss your plan as if you'd joined us.

You'll get the incident only at the start of the drill. Every candidate gets incidents of the same kind and difficulty.

How you'll be evaluated

Each panel member scores you from 1 to 5 on 7 criteria, on their own, before the panel discusses. Your final score is the weighted total. Working, well-explained depth scores higher than broad, half-finished coverage.

Criterion	Weight	Where we look	Strong (5)	Weak (1)
Running the platform reliably	30%	Tasks 3 and 5, incident drill	SLOs, alerts and an error budget policy that mean something; a proven restore; a clear diagnosis under pressure	Monitoring that exists but answers no question; restore never tested
Security and client separation	20%	Task 2, questionnaire	Flaws found and fixed, isolation proven by a test, least access, honest answers to Delta Finance	Isolation assumed, not tested; secrets still in code
Automation and scaling to many clients	15%	Task 1	Empty account to running platform in one command; a new tenant in one step	Manual steps a future hire would have to guess
Hands-on development	10%	App fixes, Task 4	Clean fixes to the app, safe deploys and migrations, a working rollback	Infrastructure changes only; app problems worked around

Criterion	Weight	Where we look	Strong (5)	Weak (1)
Ownership and judgment	10%	README, priorities, incident drill	Clear reasons for what you did first and what you left; trade-offs named	Effort spread thin with no reasoning; problems hidden
Building the reliability function	10%	Task 6	Runbooks a stranger could follow; a 90-day plan with real priorities and a hiring view	Knowledge only in your head
Communication	5%	Questionnaire, walkthrough	Plain, accurate explanations for both technical and business readers	Jargon, or claims the work doesn't support

Scores in between. A 3 means solid, competent work with clear gaps. We'll note the specific evidence behind every score.

What we won't judge: which cloud provider or tools you pick, how polished your diagrams look, or how much you spend within the cap.

After the assessment

We'll tell every candidate the outcome by [decision date]. Whatever the result, you can ask for written feedback on your work, and we'll send it. The code you write remains yours.